PrivacyGuard Security Review — Checkmarx False-Positive Justifications

Nov 9, 2026 · Author: Thangadurai Thambiannan

Summary

This document covers every finding in the Source Scanner report for PrivacyGuard 0.5.0. A first scan reported 213 findings; remediation removed 72 of them, and the scan of 0.5.0 reports 141. All 141 remaining findings are false positives, or behaviour that is intentional and controlled, and each is justified below. Line numbers refer to the 0.5.0 source exactly as scanned.

Item	Value
Package	PrivacyGuard, version 0.5.0 (04tQE00000jpLenYAE) (namespace `privacyguard`, package ID 0HoQE0000000GUb0AM)
Scan 1	Scan ID a0OKX000001JVIW2A4 (job 17421), PortalAll, Checkmarx 9.7: 213 findings
Scan 2 (current)	Scan ID a0OKX000001JVJ52AO (job 17427), PortalAll, Checkmarx 9.7, service v3.2: 111 security and 30 quality findings

Finding	Scan 1	Scan 2 (0.5.0)	Justified in this document
FLS Create (Apex CRUD Create Violation)	145 (FLS Create and Update combined)	35	35 (section 2)
FLS Update (Apex CRUD Update Violation)	(included above)	52	52 (section 2)
Sharing	4	4	4 (section 3)
Apex SOQL SOSL USER MODE MISSING	9	11	11 (section 4)
Apex CSRF In Aura LWC	9	9	9 (section 5)

Finding	Scan 1	Scan 2 (0.5.0)	Justified in this document
DML Statements Inside Loops	31	27	27 (section 6)
Queries With No Where Or Limit Clause	12	0	— (fixed)
Multiple Trigger On same sObject	3	3	3 (section 7)
Total	**213**	**141**	**141**

Controls that apply throughout

1. **Custom-permission gate.** Every Apex entry point and every asynchronous unit checks a PrivacyGuard custom permission with `FeatureManagement.checkPermission` before doing any work (`PrivacyGuardSecurityService`).
2. **Sandbox-only execution.** `EnvironmentSafetyService.assertExecutionAllowed()` refuses masking, deletion and trigger changes unless `Organization.IsSandbox` is true. It is re-checked in every batch `start()` and `execute()` and in every Tooling call. There is no production override.
3. **Private package data.** All nine PrivacyGuard custom objects have org-wide default Private.
4. **Immutable audit log.** Trigger `PGProtectMaskingLog` blocks every update and delete of `Masking_Log__c`, and replaces failure text before it is stored so no field values reach the log.
5. **Schema-validated dynamic SOQL.** Object and field names are validated against the org schema (`SchemaValidationUtil`) before they are used in a query.
6. **No external callouts.** The only callout endpoint is the subscriber's own My Domain, used to change Apex trigger status.

1. Findings fixed in code

These findings from scan 1 were fixed in 0.5.0, and scan 2 confirms they no longer appear.

Finding	Count	Location	Change
FLS_Create	8	`PIIFieldScannerBatch` 58	`insert allFindings` changed to `insert as user allFindings`

Finding	Count	Location	Change
DML in loops	11	`MaskingLogger` 65, from `PIIFieldScannerService` 57 and `MaskingJobOrchestrator` 139	Immediate log inserts inside loops replaced with `queueLog(...)` and a single `flush()`
DML in loops	2	`RuntimeMaskingJobOrchestrator` parallel launch	Per-object updates replaced with one bulk `update as system launched` (line 62)
No WHERE or LIMIT	12	`PrivacyGuardConfigurationController`, `PIIFieldScannerService`, `PrivacyGuardHomeController`	`LIMIT` clauses added; one query filtered by the requested objects
(hardening)	—	`AutomationTriggerToggleQueueable`, `MaskingFailedRecordRecoveryBatch`, `MaskingBatch`	Access modes stated explicitly (`as system`, `WITH SYSTEM_MODE`, `AccessLevel.SYSTEM_MODE`), and field edit access enforced in the masking write. These lines are still reported and are justified in sections 2 and 4.

Two further code-quality refinements (removing a defensive `completeObject` call inside the launch loop, and adding `WITH USER_MODE` to test-class queries) are scheduled for the next package version. The 22 findings they relate to are justified for 0.5.0 in sections 4 and 6.

2. FLS Create / FLS Update (87 justified)

The remaining FLS findings fall into three groups. None of them writes a subscriber business field that a user could otherwise not edit.

2.1 System-mode writes to PrivacyGuard's own state and audit objects (82)

Why this is a false positive. These statements write only to PrivacyGuard custom objects and custom settings that record internal application state: run status, record counts, async job IDs, timestamps, trigger deployment results, bypass ownership and audit entries. Every one uses an explicit `insert as system` or `update as system`. The fields are deliberately read-only or hidden in every PrivacyGuard permission set. Giving users edit FLS on them would let a user forge run results, mark a deployment as verified, take over the bypass lock, or alter audit history. Enforcing the running user's FLS here would therefore weaken security, not strengthen it.

Compensating controls. Each write is reached only after the relevant custom permission check, and destructive paths also require a sandbox org. No value written comes directly from user input: values are computed by the package (statuses, counts, IDs, timestamps),

or are user-supplied names that have already been validated against the schema. `Masking_Log__c` entries can't be edited or deleted once written.

Class	Lines (final source)	Object written	Gate before the write	Create	Update
`RuntimeMaskingJobOrchestrator`	33, 34, 62, 125, 139, 164, 167, 194, 198, 232, 241	`Masking_Job_Run__c`, `Masking_Job_Run_Object__c`	`requireRunPermission` (`start`, and `requireRuntimeAccess` line 247) and sandbox check	2	27
`MaskingLogger`	54, 65	`Masking_Log__c`	Called only from gated services and batches	15	0
`AutomationManagementController`	145, 146, 181	`Deactivated_Automation__c`	`Use_PrivacyGuard_Metadata_Deactivation` (line 34) and sandbox check	6	5
`MaskingJobOrchestrator`	184, 193, 205, 234	`Masking_Run_Tracker__c` (hierarchy setting)	`requireRunPermission` (line 45) and sandbox check	2	7
`PrivacyGuardAuditService`	16, 20, 54	`Masking_Log__c` (audit)	Called from gated services and the audit triggers; insert-only	7	0
`AutomationTriggerToggleQueueable`	79, 111	`Deactivated_Automation__c`	Metadata-deactivation permission (line 18) and sandbox check, re-checked in every `ToolingApiClient` call	0	5
`AutomationManagerService`	36, 65, 80	`Automation_Bypass__c` (hierarchy setting)	`requireRunPermission` (line 84); `FOR UPDATE` lock and owner check	1	4
`PrivacyGuardFileInventoryController`	83	`Masking_Log__c` (deletion audit)	`Purge_PrivacyGuard_Files` (line 49), sandbox check (line 50), typed DELETE confirmation	1	0
Total				**34**	**48**

2.2 DML already executed in user mode (3)

Why this is a false positive. These statements already use Apex user-mode DML (`insert as user` / `update as user` / `delete as user`), which enforces the running user's object permissions, field-level security and sharing at run time. The scanner doesn't recognise the user-mode keyword.

Class	Line	Statement	Findings
`MaskingJobManagerService`	120	`insert as user j;` / `update as user j;` (job save)	1 (Update)
`MaskingJobManagerService`	121	`delete as user existing;` / `insert as user rows;` (job objects)	1 (Create)
`PrivacyGuardConfigurationController`	239	`if(id==null) insert as user k; else update as user k;` (keyword save)	1 (Update)

2.3 The masking write itself (2)

`MaskingBatch.cls` line 239 (scan 231): `Database.update(recordsToUpdate, false)`, reached through `NumericRangeMaskStrategy`.

Why this is a false positive. FLS is enforced explicitly before any value is assigned:

- line 164: the completion checkbox must exist and be updateable by the running user, or the chunk is rejected
- line 174: `canUpdateRecords` requires the object and the completion checkbox to be updateable
- line 185: each target field must be updateable, or that field is logged as a failure and its record isn't saved

Only records whose every configured field passed these checks are added to `recordsToUpdate`. Record sharing is bypassed on purpose (see section 3); field-level security is not. This is covered by `PrivacyGuardMaskingFlsTest`, which includes a user who can edit the checkbox but not the target field, a user with no edit access to the object, and a positive control.

3. Sharing (4 justified)

Four classes are declared `without sharing` on purpose. Every other class in the package is `with sharing`, and none uses `inherited sharing`.

Class (line)	Why record sharing must be bypassed	Controls
`MaskingBatch` (57)	Masking has to reach every record of the target object in the sandbox. Under sharing, records the running user can't see would be skipped silently and keep their real personal data. That defeats the product's purpose and is the greater privacy risk.	Run permission and sandbox check in `start()` (lines 99–100) and every `execute()` (159–160). Object and field edit access is enforced (lines 164, 174, 185). Object and field names are schema-validated. Business values are never written to package objects.
`MaskingFailedRecordRecoveryBatch` (6)	Re-masks only the records that failed in an earlier run, with the same coverage as `MaskingBatch`.	Same gates (lines 35–36, 67–68). Record IDs come only from that run's own failure logs and are type-checked against the object. Writes go through `MaskingBatch`, so the same field-level checks apply.

Class (line)	Why record sharing must be bypassed	Controls
`AutomationTriggerToggleQueueable` (2)	Runs asynchronously and must update the `Deactivated_Automation__c` tracking record created for the request, whoever owns it, to record deployment progress and the verified result.	Metadata-deactivation permission (line 18), sandbox check, remote Org ID and `IsSandbox` verification on every Tooling call. Namespaced and PrivacyGuard triggers are refused.
`PrivacyGuardAuditService` (2)	Audit entries must be written for every user action, including by users who can't create `Masking_Log__c` records themselves. Otherwise audit coverage would depend on the actor's own permissions.	Insert-only. Takes no user-supplied object or field names. Entries are immutable (`PGProtectMaskingLog`), and secret values are redacted in change details.

4. Apex_SOQL_SOSL_USER_MODE_MISSING (11 justified)

4.1 Failed-record recovery batch (6)

Location	Query	Access mode
`MaskingFailedRecordRecoveryBatch.cls` 57–63 (scan 57)	`Masking_Log__c` rows for the parent run's failed records	Now explicitly `WITH SYSTEM_MODE` (line 61)
`MaskingFailedRecordRecoveryBatch.cls` 90–95 (scan 2 line 93, 5 data-flow paths)	Target records by ID for re-masking	Now `Database.queryWithBinds(soql, binds, AccessLevel.SYSTEM_MODE)`

Why this is a false positive. Recovery has to read exactly what the original masking run attempted, so both queries run in system mode on purpose.

- **Log query.** `Masking_Log__c` is Private and read-only, and the Operator permission set has no access to it. A user-mode query would return nothing, and the rerun would silently mask nothing.
- **Record query.** This must reach the same records the original sharing-bypassed run reached, or failed records the operator can't see would keep their real data.

Compensating controls.

- Run permission and sandbox check in `start()` and `execute()`.

- The log query is scoped to one parent run ID, one object, `Status__c = 'Failed'`, and a non-null record ID.
- Each logged ID is parsed as an `Id` and accepted only if its sObject type matches the job's object. Malformed IDs are dropped, with no broad fallback query.
- The object name is validated with `SchemaValidationUtil.requireValidObject`. Field names come from saved, validated masking configuration, and the completion field is resolved through the schema (`PrivacyGuardSchema.name`).
- The ID filter uses a bind variable, so there's no string concatenation of values.
- Writes go through `MaskingBatch`, which enforces object and field edit access (section 2.3).

4.2 Test code (5)

Why these are false positives. Every one of these paths starts or ends in an `@IsTest` class. Test classes run only during deployment and test execution, under the test-running administrator. Subscribers can't call them, and they read only data the test itself created. No runtime query is involved.

Source → sink	What the query does
`PIIFieldScannerBatch` 79 → `PIIScanReviewControllerTest` 69	Test assertion reading the test batch's `AsyncApexJob` status
`PIIFieldScannerBatch` 79 → `PIIScanReviewControllerTest` 72	Test assertion counting the scan results the test produced
`PrivacyGuardMaskingFlsTest` 22 → 45	Test setup looking up a standard `Profile` to create a restricted test user
`RegexMaskSafetyTest` 70 → `PrivacyGuardSchema` 20	Test query loading a Contact the test inserted; the sink returns describe metadata (field names), not record data
`PrivacyGuardMaskingFlsTest` 49 → `PrivacyGuardSchema` 20	Same pattern as above, in the field-level security test

The next package version adds `WITH USER_MODE` to these test queries.

5. Apex_CSRF_Aura_LWC (9 justified)

Why this is a false positive. Every flagged Apex method that changes data is called imperatively from an LWC event handler bound to an explicit user action: a button click or a confirmation dialog. None is called from `connectedCallback`, `renderedCallback` or a

`@wire` adapter. Lightning components also send every Apex request through the Aura framework with its platform anti-CSRF token. A cross-site request can't invoke these methods.

Compensating controls.

- Each method checks its PrivacyGuard custom permission on the server.
- Destructive methods also require a sandbox org.
- File deletion additionally requires the literal confirmation `DELETE` (checked on the server) and at most 100 IDs per request.

LWC call (file:line)	User action that triggers it	Apex method (line)	Server-side checks
`dataMaskingConfiguration.js` 126	**Save** button (`saveExisting`)	`PrivacyGuardConfigurationController.saveConfigurations` (62)	Manage PII permission; strategy and pattern validation; user-mode DML
`dataMaskingConfiguration.js` 143	**Add selected** button (`addSelected`)	`saveConfigurations` (62)	As above
`dataMaskingConfiguration.js` 145	**Delete** confirmation dialog (`confirmDelete`)	`deleteConfigurations` (166)	Manage PII permission; `delete as user`
`piiKeywordConfiguration.js` 26	**Add Keyword** button (`add`)	`saveKeyword` (226)	Manage PII permission; `insert as user`
`piiKeywordConfiguration.js` 36	Row **Save** button (`saveRow`)	`saveKeyword` (226)	Manage PII permission; `update as user`
`piiKeywordConfiguration.js` 44	Row **Delete** button (`removeRow`)	`deleteKeyword` (242)	Manage PII permission; user-mode DML
`piiScanReview.js` 94	**Accept / Remove Selected**, **Accept / Remove Object** buttons, with confirmation	`PIIScanReviewController.updateReviewStatus` (102)	Manage PII permission; `update as user`
`privacyGuardFileInventory.js` 39	**Delete** after typing DELETE in the confirmation dialog (`confirmDelete`)	`PrivacyGuardFileInventoryController.deleteSelected` (48)	Purge permission, sandbox check, server-side `DELETE` confirmation, 1–100 IDs, `AccessLevel.USER_MODE` delete
`automationManagement.js` 13	**Check status** button (`checkDeployments`)	`AutomationManagementController.resumeDeploymentChecks` (169)	Metadata-deactivation permission; sandbox check; only re-queues status checks for up to 20 existing "Check required" records the user can see (`WITH USER_MODE`); submits no new deployments

Disclosure: reconciliation when the masking control panel loads

Checkmarx did not flag this, but we're disclosing it. When the Masking Jobs view loads, `RuntimeMaskingJobOrchestrator.reconcile` (line 68) can update run records. It closes run objects whose batch job has already ended without calling `finish()` , and finalises the run. This is safe on a page load:

- it takes no input except a job ID the user can already see (`WITH USER_MODE` query)
- it is idempotent, and writes nothing when there is nothing to reconcile
- it can only move a run to a terminal state that matches the platform's own `AsyncApexJob` status
- it changes no business data and starts no new work
- it requires the Run Masking permission (`requireRuntimeAccess`)

6. DML_Statements_Inside_Loops (27 justified)

Why these are false positives. Every path falls into one of three cases:

- the DML isn't inside a loop at all, and Checkmarx flags it because the list being written was built in an earlier loop
- the DML runs once per transaction or per batch chunk
- the loop is bounded by the objects in a single job, and the branch can't run in practice

Two paths start in test code. None of them can approach the 150-DML governor limit.

Sink (0.5.0 line)	Path reported	Why it's bounded or not a loop	Findings
`RuntimeMaskingJobOrchestrator` 62, 194, 198, 232, 241, 242, via line 64	`start` / `completeObject` → `launchAll` loop (line 52) → `completeObject(ro.Id, …)` (line 64)	Line 64 is a defensive branch for run objects with no active masking configuration. `start()` rejects any such object in its preflight (line 26), in the same transaction and before `launchAll` runs, so the branch can't run during a start. On the serial path, `launchAll` receives a single object, so the loop runs at most once. It's bounded by the objects in one job either way. The next version removes the loop (see section 1).	17
`RuntimeMaskingJobOrchestrator` 125, 139, 140	`MaskingJobManagerService` 90 → `reconcile` (line 68)	`reconcile` handles a single run (`LIMIT 1`). Run objects are updated once, in bulk (line 125), after the loop that builds the list. The run and job updates at 139–140 run once per call.	3
`RuntimeMaskingJobOrchestrator` 62	`start` builds run objects (line 34) → `launchAll`	A single bulk `update as system launched` after the launch loop: the list is built in the loop, and the DML is outside it	1

Sink (0.5.0 line)	Path reported	Why it's bounded or not a loop	Findings
`AutomationManagementController` 145, 146	`PrivacyGuardConsolidatedSecurityTest` 41 → `queueChanges`	Path starts in a test. At runtime, one bulk `update/insert as system changes` runs after the validation loop (lines 121–140 build the list).	2
`MaskingBatch` 239	Recovery batch → `MaskingBatch.execute`	One `Database.update` per batch chunk, which is the standard Batch Apex pattern	2
`PrivacyGuardAuditService` 16	`MaskingJobManagerService` 197 (cancel schedule)	Loop over the job's scheduled `CronTrigger` rows, filtered to two fixed names (`PGWeekly_<id>` , `PGSchedule_<id>`), so at most 2 audit inserts	1
`AutomationManagerService` 36	`PrivacyGuardInternalAuthorizationTest` 51	Path starts in a test method. The runtime call updates the single org-default setting row once.	1

7. Multiple_Trigger_On_same_sObject (3 justified)

Why this is a false positive. Each flagged trigger is the only trigger in the package on its object, and the three are on three different objects. There is no second trigger on any of them whose execution order could conflict.

Trigger	Object	Events	Other PrivacyGuard triggers on this object
`PGAuditMaskingFieldConfig`	`Masking_Field_Config__c`	after insert, update, delete, undelete	None
`PGAuditMaskingJob`	`Masking_Job__c`	after insert, update, delete, undelete	None

Trigger	Object	Events	Other PrivacyGuard triggers on this object
`PGAuditMaskingJobObject`	`Masking_Job_Object__c`	after insert, update, delete, undelete	None

The package's fourth trigger, `PGProtectMaskingLog`, is on a fourth object, `Masking_Log__c`. All four are thin: each passes `Trigger.new` / `Trigger.old` to `PrivacyGuardAuditService` in bulk, or validates in a single loop with no SOQL or DML.